

RaViLa 개발 경과 보고서 v2 — 학습편

SFT 대규모 학습 → RLVR/GRPO · 문제점·근거·변경사항

이전 보고서(RaViLa_report.pdf) 이후 구간 · 8B Qwen3-VL · 5 × NVIDIA B200

이 문서의 모든 수치는 디스크의 산출물에서 읽어 생성했습니다. runs/08_grpo_eval/*.json, runs/09_structure/*.json, runs/04_diagnostics/pipeline_*.json, 학습 로그. 아직 끝나지 않은 실행은 추정치를 넣지 않고 '진행 중'으로 표시했습니다.

이 보고서의 결론 한 줄

이전 보고서의 핵심 수치가 오염된 계기로 측정된 것이었습니다. 레이더 기여도 +0.539는 '모델이 레이더를 읽는다'는 뜻으로 보고됐지만, 실제로는 제가 인코더에게 토큰에 적으라고 지도학습시킨 세 개의 스칼라를 언어모델이 되읽는 것을 재고 있었습니다. 인코더가 배우지 않은 질문으로 다시 재면 같은 모델의 기여도가 +0.06입니다.

이 발견이 지난 두 보고서 구간에서 7가지 표현 개입, 25배 스케일업, 4가지 GRPO 보상이 모두 아무것도 움직이지 못한 이유를 설명합니다. 움직이려던 지표는 이미 천장에 붙어 있었고, 정작 0인 부분은 측정되지 않고 있었습니다.

보고서는 이 결론에 도달하기까지의 순서를 그대로 따릅니다. 중간에 제가 틀렸다고 보고한 지점들도 지웠다 다시 쓰지 않고 남겨 두었습니다 — 어떤 근거로 무엇을 믿었다가 왜 뒤집었는지가 검토에 필요한 정보이기 때문입니다.

0. 용어

본문에서 반복되는 용어를 먼저 정의합니다. 이미 아시는 항목은 건너뛰셔도 됩니다.

0.1 측정 용어

용어	정의	왜 이 프로젝트에 필요한가
shuffled 대조군	같은 모델에 다른 클립의 레이더를 넣고 같은 질문을 던진 것	정답을 카메라만으로 맞힐 수 있으면 full 점수가 높아도 레이더를 쓴 게 아닙니다. 그 몫을 빼기 위한 통제입니다.
레이더 기여도	full 상관 - shuffled 상관	이 보고서의 주 지표. 0이면 그 태스크는 레이더 없이 풀리고 있습니다.
상관계수 r	예측 숫자와 정답 숫자의 선형 상관. -1~+1	정답의 스케일이 태스크마다 달라(개수 ~700, 각도 ~10도) 절대오차보다 비교가 쉽습니다.
질문형별 분리 (by_form)	한 태스크 안에서 질문 종류별로 나눠 상관을 계산	스케일이 100배 다른 질문을 섞으면 상관계수가 '어떤 질문이었나'만 감지해 1.0에 가까워집니다. §3의 버그가 바로 이것입니다.
시드 노이즈 바닥	설정을 고정하고 난수 시드만 바꿔 반복했을 때의 표준편차	개입 효과가 이 폭보다 작으면 '차이 없음'입니다. §5에서 이 프로젝트의 값을 실측했습니다.
오염도 (contamination)	프로브 정답이 지도학습된 스칼라와 갖는 최대 상관	이 값이 크면 그 프로브는 모델이 스칼라만 읽어도 풀립니다. §7에서 새 프로브마다 실측했습니다.

0.2 학습 용어

용어	정의
SFT (supervised fine-tuning)	정답 텍스트를 그대로 교사로 삼아 다음 토큰을 맞히도록 학습. 이 프로젝트의 기본 학습.
교차 엔트로피 손실	정답 토큰의 로그 확률. 숫자 638을 639로 틀리든 100으로 틀리든 똑같이 벌합니다 — '얼마나 틀렸나'를 표현할 수 없는 것이 RLVR을 도입한 이유입니다.
RLVR (verifiable rewards)	사람 선호가 아니라 프로그램이 채점할 수 있는 보상으로 강화학습. 정답이 라벨에서 계산된 숫자이므로 채점기가 정확히 맞고 틀림을 압니다.
GRPO	한 프롬프트에서 G개를 샘플링하고 그 그룹 안에서 보상을 정규화해 이득(advantage)을 만드는 방식. 가치망(critic)이 필요 없어 8B 모델을 하나 더 안 띄웁니다.
advantage / spread	그룹 내 보상 편차. 이것이 0이면 모든 샘플이 같아 학습 신호가 사라집니다. 본문의 spread는 이 값입니다.
clipped ratio	새 정책과 샘플링 당시 정책의 확률 비를 ± 0.2 로 자르는 것. 한 번의 갱신이 폭주하지 않게 막습니다. 참조 모델 사본이 따로 필요 없습니다.
catastrophic forgetting	새 태스크를 학습하면서 이전 능력이 무너지는 현상. §5에서 SFT 대조군이 정확히 이것을 보였습니다.

0.3 모델 구조 용어

용어	정의
레이더 인코더	포인트클라우드(점마다 거리·방위각·RCS·도플러)를 240개 토큰으로 압축하는 트랜스포머. 언어모델은 원시 점을 보지 않고 이 토큰만 봅니다.
head_stats	인코더 출력 토큰에서 5개 통계량(log_n_points, log_n_moving, max_rcs, mean_rcs, max_range)을 예측하게 하는 보조 헤드. 6장의 문제의 원인입니다.
커넥터	인코더 토큰을 언어모델의 임베딩 차원으로 옮기는 선형층.
forward hook 주입	레이더 토큰을 임베딩 자리에 밀어 넣는 방식. inputs_embeds를 쓰면 Qwen3-VL의 비디오 토큰 배치가 꺼져 버려서 흑을 쓸 수밖에 없었습니다.
FSDP2	모델 파라미터를 GPU들에 쪼개 두는 분산 학습(샤딩). 8B 전체 미세조정을 LoRA 없이 하기 위한 것입니다.

1. 출발점 — 이전 보고서가 끝난 지점

이전 보고서의 결론은 두 가지였습니다.

① **진단**: 레이더 정보는 언어모델 은닉 상태까지 도달한다. 4지점 프로브 결과:

체크포인트	인코더	커넥터	은닉 상태	은닉(shuffled)
vlm_8B_align_fixed	0.670	0.685	0.654	0.008
vlm_8B_full_resume	0.673	0.686	0.650	0.003
vlm_8B_sw_cmp_base	0.672	0.687	0.656	0.009

은닉 상태에서 R^2 0.65로 복원되고, 남은 레이더를 넣으면 0.008로 무너집니다. 즉 정보는 있고 레이더에서 온 것이 맞습니다. 그런데 모델이 실제로 쓴 숫자는 그보다 훨씬 낮은 상관을 보였습니다. 표현이 아니라 목적함수가 문제라는 진단이었습니다.

② **실패한 개입**: 그 진단에 근거해 손실을 고쳤으나 통제군 5개를 갖춘 검정에서 효과가 확인되지 않았습니다($p=0.152$).

이 진단에는 §6에서 드러날 결함이 있습니다. 프로브가 복원한 양이 `max_rcs`인데, 그것이 바로 인코더가 지도학습으로 토큰에 적어 넣도록 배운 값입니다. '정보가 도달한다'는 관찰은 사실이지만 놀라운 일이 아니었고, '그러므로 표현은 충분하다'는 추론은 그 세 스칼라에 한해서만 성립합니다.

2. 테스트 집합 분리

이전 보고서 시점에는 QA 태스크에 전용 테스트 집합이 없었습니다. `datatools/make_qa_holdout.py`로 1,999개 QA 클립 중 99개를 sha1 해시로 골라 학습에서 제외했습니다(1,940문항). 해시로 고른 이유는 재현성입니다 — 같은 클립 목록에서 항상 같은 99개가 나옵니다.

분할은 클립 단위입니다. 같은 클립의 프레임이 학습과 테스트에 나뉘어 들어가면 같은 장면을 외운 것을 일반화로 오인하게 됩니다.

남은 한계: 현재 8B 기준 모델 `vlm_8B_long_base`은 이 홀드아웃이 적용되기 전에 학습됐습니다. 따라서 이 모델의 QA 점수는 테스트 점수로 쓸 수 없습니다. 다음 전체 재학습부터 유효해집니다.

3. 평가에서 발견해 고친 문제 5건

학습을 늘리기 전에 **측정부터 고쳐야 했습니다**. 아래는 전부 '결과가 좋아 보이게 만들던' 버그이고, 고친 뒤 수치가 내려갔습니다.

#	문제	증상	수정
1	질문형 pooling (radar_probe)	세 질문의 답 크기가 100배 차이라, 상관계수가 '어떤 질문이었나'만 감지해 0.994를 기록. 모델이 아무것도 추적하지 않아도 나오는 값.	질문형별로 분리해 각자 스케일에서 채점
2	질문형 pooling (radar_transfer)	거리 질문이 '...of its detections?'로 끝나 개수 질문과 같은 형식으로 분류됨. 미터(~88)와 개수(~200)를 한 상관에 혼합.	구문 매칭 표로 교체. 10개 질문형 전수 검증
3	shuffled가 무효	배치 1에서 roll(shifts=1)은 자기 자신을 가리키는 무연산. full과 shuffled가 소수점 4자리까지 동일했음.	직전 아이템의 레이더를 실제로 들고 오도록 변경
4	매칭 임계값	task 02는 방위각이 없는데 (x,y) 2 m 임계로 채점.	거리 전용 1 m 임계 분리
5	릿지 프로브 penalty	고정 penalty로 R ² -0.5가 나온 양이, 홀드아웃으로 alpha를 튜닝하니 +0.67.	분할 검증으로 alpha 선택

1번의 효과 크기: 같은 모델·같은 데이터에서 pooled 상관 **0.994** → 질문형별 **0.724**. 즉 이전 보고서 이전의 평가 수치는 0.27만큼 부풀려져 있었습니다.

2번은 이번엔 §7 작업 중 발견했습니다. 1번과 같은 종류의 버그가 다른 태스크에 남아 있던 것입니다. 따라서 본 보고서의 radar_transfer 수치는 재측정 대상이며, 아래 표의 값은 수정 전 기준입니다.

4. 11개 태스크 전체 평가 체계

태스크마다 정답의 형태가 달라 하나의 지표로 썰 수 없습니다. training/task_scorers.py에 태스크별 채점기를 두고 training/eval_all_tasks.py가 생성 → 채점 → shuffled 대조를 수행합니다.

채점기	적용 태스크	지표
score_objects	det_objects, track_identity, motion_seg	탐지 F1 (greedy nearest-first 매칭, 2 m)
score_waypoints	plan_ego	변위 MAE (m)
score_trajectory	agent_traj	거리 MAE (m), 방위각 MAE (deg)
score_quantity	radar_probe, radar_transfer, depth_range, world_model, radar_structure	질문형별 상관 + shuffled 대조
score_tags	retrieval	태그 집합 F1
score_choice	qa	정답률
score_text	desc_*	손실만 (참조 없는 문장 점수는 정적하지 않음)

SFT 300k 기준 모델의 전체 태스크 결과입니다 (runs/06_task_report/, val, 태스크당 200문항).

태스크	지표	full	shuffled
det_objects	F1	0.060	0.063
radar_probe	상관	0.994	0.856
track_identity	F1	0.365	0.366
radar_transfer	상관	0.826	0.554
motion_seg	F1	0.049	0.054
retrieval	F1	0.593	0.600
desc_radar	손실만	--	--
plan_ego	변위 MAE (m)	1.12	1.15
agent_traj	거리 MAE (m)	5.08	5.30

태스크	지표	full	shuffled
desc_complementarity	손실만	--	--
depth_range	상관	0.211	0.215
world_model	상관	0.260	0.315

이 표는 §6의 발견 이전 기준입니다. radar_probe의 높은 상관은 지도학습된 스칼라의 되었기이고, det_objects의 낮은 F1은 2 m 매칭 임계가 엄격한 탓이 큽니다. 즉 이 표만으로 태스크별 강약을 판단하면 안 됩니다 — 각 수치가 무엇을 재는지는 §3·§6·§7을 함께 읽어야 합니다.

5. RLVR / GRPO 도입

5.1 왜 강화학습인가

교차 엔트로피는 **얼마나 틀렸는지를 표현할 수 없습니다**. 정답 638에 대해 639라고 답하든 100이라고 답하든, `argmax`가 틀렸다는 사실만 같은 크기로 벌합니다. 그런데 이 프로젝트의 레이더 태스크는 전부 숫자를 묻습니다. 그리고 §1의 진단은 **정보가 이미 은닉 상태에 있다고** 말합니다. 즉 '강화학습이 없는 정보를 만들어낼 수는 없다'는 일반적 반론이 이 경우에는 적용되지 않는다는 것이 도입 근거였습니다.

보상은 라벨에서 계산된 숫자와 비교하는 것이므로 **사람도 선호 모델도 필요 없습니다**. 이것이 RLHF가 아니라 RLVR인 이유입니다.

5.2 구현

가치망 없는 GRPO를 택했습니다. PPO를 그대로 쓰면 8B 크기의 `critic`이 메모리에 하나 더 올라갑니다. 그룹 내 정규화가 그 역할을 대신하고, 참조 모델도 두지 않았습니다 — 샘플링 당시 정책에 대한 `clipped ratio`가 폭주를 막습니다.

보상	정의	의도
relative	$1 - \text{예측} - \text{정답} / \max(\text{정답} , 1)$, 0에서 절단	가까울수록 높은 연속 보상
exact	정확히 일치할 때만 1	채점의 세밀함이 필요한지 보는 대조군
tolerance	정답의 $\pm 10\%$ 안이면 1	정밀도는 관대하되 여전히 이진
all_numbers	답 안의 모든 숫자를 각각 relative로 채점해 평균	<code>radar_probe</code> 는 두 수치를 한 번에 묻는데 첫 숫자만 채점하면 두 번째가 방치됨

5.3 멀티모달 GRPO에서 고친 버그 2건

증상	원인	수정
<code>generate(num_return_sequences=G)</code> 에서 텐서 크기 불일치	레이더 토큰은 배치 1인데 생성은 G개를 동시에 만듦	배치 1이면 G로 broadcast
3-D rope 위치 계산에서 예외	<code>mm_token_type_ids</code> 는 프롬프트 길이인데 프롬프트+생성문을 채점하면 더 김. Qwen3-VL이 이걸 attention mask로 인덱싱	생성 구간을 0(텍스트)으로 패딩해 길이를 맞춤

5.4 소규모 GRPO 결과 (800 스텝)

실행	radar_probe 기여	radar_transfer 기여	depth_range full	depth_range 기여
SFT 300k (출발점)	+0.539	+0.365	0.200	-0.048
SFT 계속 (대조군)	+0.497	-0.000	0.314	-0.131
GRPO relative s0	+0.568	+0.356	0.423	-0.049
GRPO relative s1	+0.542	+0.248	0.408	-0.032
GRPO relative s2	+0.512	+0.264	0.439	+0.145
GRPO all_numbers	+0.560	+0.355	0.430	-0.064
GRPO exact	+0.555	+0.375	0.398	-0.075
GRPO tolerance	+0.542	+0.377	0.471	+0.010

확인된 효과 ①: GRPO는 파국적 망각을 막습니다. 같은 데이터를 SFT로 계속 학습한 대조군(강조 행)은 radar_transfer가 0.838 → 0.166로 무너졌고 레이더 기여도가 정확히 0이 되었습니다. GRPO 여섯 실행은 전부 0.85 근처를 지켰습니다. 이 차이는 시드 폭(\$5.5)을 크게 넘으므로 실재합니다.

확인되지 않은 것: GRPO는 레이더 기여도를 올리지 못했습니다. GRPO 실행들의 +0.51~+0.57은 출발점인 SFT 모델에 이미 있던 값입니다(SFT 300k = +0.539). 제가 사전에 relative가 exact를 이길 것으로 예측했으나 틀렸습니다.

5.5 시드 노이즈 바닥 — 위 표를 어떻게 읽어야 하는가

태스크	시드 3개 값	평균	표준편차	폭
radar_probe	+0.568, +0.542, +0.512	+0.541	0.028	0.056
radar_transfer	+0.356, +0.248, +0.264	+0.289	0.058	0.108

보상 네 종류의 차이는 전부 이 폭 안에 있습니다. radar_probe에서 보상 간 폭은 0.026인데 시드 폭이 0.056입니다. radar_transfer는 더 심해서 보상 간 0.022 대 시드 0.108, 약 5배 차이입니다.

즉 보상 선택은 이 실험 설계로 구분 자체가 불가능합니다. '차이가 없다'가 아니라 '있어도 못 져다'입니다. 이 시드 바닥을 확보하기 전에는 제가 보상 비교표를 결과로 보고했었고, 그것은 **철화합니다**.

정정 ②: depth_range가 0.200 → 0.42~0.47로 두 배가 됐다고 보고했으나, 그것은 full 상관이었습니다. shuffled도 함께 올라 기여도는 여전히 0 근처입니다. 태스크 정확도는 실제로 좋아졌지만 그건 카메라 몫입니다.

6. 가장 큰 발견 — 계기가 오염돼 있었습니다

6.1 어떻게 드러났는가

검토 중 다음 질문을 받았습니다. "QA를 풀 때는 정답이 프롬프트에 들어가면 안 되는데, probe는 넣은 것 아니냐?" 텍스트 프롬프트에는 정답이 없으므로 처음에는 문제없다고 판단했습니다. 그러나 확인해 보니 **인코더 쪽에 있었습니다.**

```
STATISTICS = ("log_n_points", "log_n_moving", "max_rcs", "mean_rcs", "max_range")
# head_stats: 인코더가 내보내는 토큰에서 이 5개를 예측하도록 지도학습
```

그런데 radar_probe가 묻는 것이 **detection 수, moving 수, max_rcs** — 앞의 세 개와 정확히 같습니다. 즉 이 태스크는 '모델이 레이더를 이해하는가'가 아니라 **'인코더에게 적으라고 시킨 숫자를 언어모델이 읽는가'**를 재고 있었습니다.

6.2 질문형별로 쪼갬 증거

실행	detections (지도학습됨)	rsc (지도학습됨)	illuminated (지도학습 안 됨)
SFT 300k (출발점)	+0.843	+0.818	-0.044
SFT 계속 (대조군)	+0.679	+0.734	+0.078
GRPO relative s0	+0.845	+0.816	+0.045
GRPO relative s1	+0.897	+0.673	+0.057
GRPO relative s2	+0.905	+0.596	+0.036
GRPO all_numbers	+0.837	+0.831	+0.010
GRPO exact	+0.836	+0.808	+0.022
GRPO tolerance	+0.846	+0.756	+0.024

6개 실행에서 예외 없이 같은 패턴입니다. 지도학습된 두 형식은 +0.60~+0.91, 지도학습되지 않은 illuminated는 -0.04~+0.06입니다. illuminated는 레이더 반사점을 물체 박스에 연결해야 답할 수 있어 스칼라로는 풀리지 않는 질문입니다.

따라서 헤드라인으로 보고해온 **+0.539는 스칼라 되읽기**이고, 구조적 레이더 이해를 보여주는 증거는 **없습니다.**

6.3 왜 인코더를 고치지 않는가

head_stats를 빼면 됩니다 — 그러나 뺄 수 없습니다. 그것이 없을 때 readout과 temporal stack이 gradient를 아예 받지 못해 **72개 파라미터의 grad가 None**이었고, 인코더가 학습되지 않았습니다. 이 프로젝트 초반에 고친 버그가 정확히 그것입니다. 즉 head_stats는 학습에 필요하고, 문제는 **그것을 그대로 평가에 쓴 것**입니다. 고칠 곳은 인코더가 아니라 계기입니다.

7. 새 계기 — radar_structure

인코더가 배우지 않은 질문 5종으로 프로브를 만들었습니다. datatools/radar_structure.py · 469,314개, 156,457 클립 (train 235,047 / val 141,906 / test 92,361).

학습에는 쓰지 않습니다. ALL_TRAIN_TASKS에서 제외했습니다. 학습에 넣으면 또 하나의 외운 뼈대가 되어 유일한 정직한 계기를 잃습니다.

form	질문	오염도	판정
bearing	가장 가까운 반사점의 방위각	0.035	깨끗
lateral	중심 왼쪽 반사점의 비율(%)	0.123	깨끗
spread	최좌·최우 반사점 사이 방위각 폭	0.458	부분적
closing	가장 빠르게 접근하는 반사점의 속도	0.562	부분적
near_far	20 m 이내 반사점의 비율(%)	0.690	부분적

오염도는 프로브 정답이 지도학습된 세 스칼라와 갖는 최대 |상관|이며, 92k개 테스트 프로브에서 실측한 값입니다. 이 표는 보고서를 생성할 때마다 다시 계산됩니다 — 누가 질문을 바꿔 더러워지면 표가 바뀝니다.

7.1 만드는 도중 스스로 잡은 오염

처음에는 lateral을 '왼쪽 몇 개, 오른쪽 몇 개'로 물었습니다. 채점기가 첫 숫자를 쓰는데 그것이 n_left이고, 이 값은 전체 포인트 수와 함께 커집니다. 그리고 전체 포인트 수는 지도학습된 값입니다. 실측 오염도 $r = 0.84$ 였고, SFT 모델이 이 프로브에서 기여도 +0.999를 기록했습니다.

이것을 잡지 못했다면 '레이더 공간 구조를 이해한다, 기여도 +0.999'라는 정반대 결론을 보고할 뻔했습니다. 백분율로 바꾸니 오염도가 0.12로 떨어졌고, near_far도 같은 이유로 백분율로 바꿨습니다.

7.2 결과 — SFT 300k 기준 모델

form	오염도	full	shuffled	레이더 기여
bearing	0.035	-0.032	-0.100	+0.068
lateral	0.123	0.016	-0.038	+0.054
spread	0.458	0.302	0.227	+0.076
closing	0.562	0.371	0.200	+0.170
near_far	0.690	0.303	0.090	+0.213
전체	--	0.192	0.076	+0.116

깨끗한 두 프로브(강조 행)의 기여도가 +0.054, +0.068입니다 — 즉 0입니다. 부분 오염된 세 프로브가 더 높게 나오는 것도 오염도 순서와 일치합니다. 어느 쪽으로 읽어도 결론은 같습니다: 각도·공간 구조는 언어모델에 도달하지 않습니다.

같은 모델을 radar_probe로 재면 +0.539, radar_structure로 재면 +0.116입니다. 차이는 모델이 아니라 계기입니다.

8. 대규모 GRPO 실행

8.1 설계와 근거

결정	근거
보상 = all_numbers	네 보상이 시드 노이즈 안에서 구분되지 않으므로 (§5.5) 보상 모양은 지렛대가 아닙니다. 그렇다면 답의 모든 숫자를 채점하는 쪽이 완전합니다.
3개 태스크 동시 (probe/transfer/depth)	GRPO에서 확인된 유일하게 확실한 효과가 '다른 태스크를 안 망가뜨림'이므로, 그 강점은 여러 태스크를 함께 학습할 때만 값을 합니다.
단일 태스크 대조군	'다중 태스크'와 'GRPO'를 분리해 검증하기 위해. 이것이 없으면 §5.4에서 저지른 것과 같은 착각(이미 있던 값을 성과로 오인)을 반복합니다.
같은 규모 SFT 대조군	소규모에서 파국적 망각을 보였는데 대규모에서도 그런지 확인.
온도 1.2 arm	4,000 스텝은 이전의 5배. spread가 0이 되면 advantage가 0이 되어 학습이 조용히 멈추는데 손실은 정상으로 보입니다. 그 판별용.
시드 2개	§5.5에서 실측한 시드 폭 때문에 최소 2개가 필요합니다.

8.2 진행 상황

arm	진행	역할
multi_s0	4000 / 4000	본 실험 (3태스크, 시드 0)
multi_s1	4000 / 4000	본 실험 (3태스크, 시드 1)
sftcontrol	790 / 799	SFT 대조군 (같은 3태스크)
single_s0	4000 / 4000	단일 태스크 대조군
temp12	4000 / 4000	탐색 붕괴 판별 (온도 1.2)

판정 기준을 미리 고정합니다. §5.5의 시드 폭에 따라 radar_probe는 0.06 이상, radar_transfer는 0.11 이상 벌어져야 믿을 수 있습니다. 그 아래면 '차이 없음'으로 보고합니다. 그리고 결과가 어느 쪽이든 §7의 계기로도 함께 채점합니다 — radar_probe 점수만으로는 §6의 함정을 반복하게 됩니다.

8.3 진행 중 정정한 판단

스텝 80에서 '단일 태스크 arm이 붕괴 중'이라고 보고했으나 **틀렸습니다**. spread는 스텝마다 프롬프트 2개로만 재기 때문에 크게 흔들리고, 구간 평균으로 다시 재니 단일 태스크는 0.100 → 0.092 → 0.106으로 **평평**했습니다. 한 스텝의 값으로 추세를 말한 것이 잘못이었습니다.

9. 지금까지의 정리와 다음 단계

9.1 확인된 것

주장	근거	신뢰도
GRPO는 SFT의 파국적 망각을 막는다	SFT 대조군 radar_transfer 0.838 → 0.166, GRPO 6개 실행 전부 0.85 근처 유지	높음 (효과가 시드 폭의 6배)
보상 모양은 효과가 없다	보상 간 폭 0.026 대 시드 폭 0.056	높음 (구분 불가가 결론)
레이더의 스칼라 통계는 전달된다	지도학습된 두 질문형에서 기여 +0.60~+0.91	높음 (단, 순환 측정)
레이더의 공간 구조는 전달되지 않는다	깨끗한 프로브 3종(illuminated, bearing, lateral) 전부 기여 0	높음 (독립적 프로브 3개 일치)
GRPO가 레이더 기여도를 올린다	올리지 못했음. +0.55는 출발 SFT 모델에 이미 있던 값	반증됨
depth_range가 전이로 좋아진다	full은 올랐으나 shuffled도 같이 올라 기여는 0	반증됨

9.2 다음에 해야 할 일

- ① 대규모 실행 5 arm을 11개 태스크 + radar_structure로 채점. radar_transfer는 §3의 pooling 수정판으로 재측정.
- ② 인코더 토큰이 각도 정보를 담고 있는지 프로브로 확인. 지금까지의 결과는 '언어모델이 각도를 출력하지 못한다'까지만 보이고, 그것이 (a) 인코더가 각도를 버렸기 때문인지 (b) 언어모델이 못 읽는 것인지 구분하지 못합니다. §1의 4지점 프로브를 max_rcs가 아니라 **방위각으로 다시 돌리면** 걸립니다. 이것이 다음 개입의 위치를 정합니다.
- ③ (a)로 판명되면 head_stats에 각도 항을 추가하는 것이 자연스러운 후보입니다. 다만 **그때는 radar_structure를 평가에서 빼야 합니다** — 지도학습하는 순간 그 프로브도 §6과 같은 순환 측정이 됩니다. 그래서 프로브 5종 중 일부만 지도학습하고 나머지를 계기로 남기는 설계가 필요합니다.
- ④ QA 홀드아웃이 적용된 전체 재학습. 현재 기준 모델은 그 이전 학습이라 QA 테스트 점수를 쓸 수 없습니다(§2).

9.3 이 보고서에서 철회하거나 정정한 항목

이전 보고	정정
레이더 기여도 +0.539 = 모델이 레이더를 읽는다	지도학습된 스칼라의 되읽기. 정직한 계기로는 +0.06
보상별 성능 비교표 (relative/exact/tolerance/all_numbers)	전부 시드 노이즈 안. 비교표 철회
relative가 exact보다 나올 것이라는 예측	틀렸음. 구분되지 않음
depth_range가 0.200 → 0.47로 두 배	full 기준. shuffled도 올라 기여도는 0
GRPO 대규모로 레이더 기여도를 올릴 수 있다	근거 없음. 대신 망각 방지 효과를 목표로 재설정
단일 태스크 arm이 붕괴 중 (스텝 80)	한 스텝의 값이었음. 구간 평균은 평평
radar_transfer 기여 +0.365	두 질문형이 pooling된 상태의 값. 재측정 필요

보고서 생성: `python -m datatools.build_report_v2`